

Dokumentacja projektowa Konsolowy system zgłaszania usterek

Pole	Wartość
Nazwa projektu	Konsolowy system zgłaszania usterek
Przedmiot	PBL - Project Based Learning
Technologia	C++ / aplikacja konsolowa
Plik danych	zgłoszenia.txt
Autorzy	Bartłomiej Strączek, Marcel Kret

Dokument opisuje założenia, zakres, strukturę, sposób działania oraz harmonogram realizacji aplikacji konsolowej służącej do rejestrowania i obsługi zgłoszeń usterek.

Spis treści

- 1. Streszczenie projektu
- 2. Cel i uzasadnienie projektu
- 3. Zakres systemu
- 4. Wymagania funkcjonalne i нефункционалне
- 5. Mapa myśli projektu
- 6. Przypadki użycia
- 7. Model danych
- 8. Architektura i opis kodu źródłowego
- 9. Przepływ działania programu
- 10. Wykres Gantta i harmonogram
- 11. Plan testów
- 12. Analiza ryzyka
- 13. Instrukcja uruchomienia
- 14. Możliwości rozwoju
- 15. Podsumowanie

1. Streszczenie projektu

Projekt dotyczy wykonania prostej aplikacji konsolowej w języku C++, której zadaniem jest obsługa zgłoszeń usterek. Program umożliwia dodawanie nowych zgłoszeń, przeglądanie listy zgłoszeń, wyszukiwanie zgłoszenia po numerze ID, zmianę statusu, usuwanie pozycji oraz prezentację statystyk. Dane są przechowywane w pamięci programu w kontenerze vector, a trwałość danych zapewnia zapis do pliku tekstowego zgloszenia.txt.

Projekt został zaprojektowany jako rozwiązanie edukacyjne: jest czytelny, możliwy do rozbudowy i pokazuje podstawowe elementy programowania strukturalnego oraz pracy z plikami. Aplikacja nie wymaga bazy danych ani interfejsu graficznego, dzięki czemu zakres jest odpowiedni dla projektu PBL realizowanego w formie aplikacji konsolowej.

2. Cel i uzasadnienie projektu

Celem projektu jest stworzenie programu, który symuluje prosty system techniczny wykorzystywany np. w szkole, akademiku, firmie lub budynku administracyjnym. W praktyce takie systemy porządkują komunikację między osobą zgłaszającą problem a osobą odpowiedzialną za jego rozwiązanie. Nawet uproszczona wersja konsolowa pozwala przedstawić pełny cykl obsługi zgłoszenia: od rejestracji, przez zmianę statusu, aż po zamknięcie lub usunięcie wpisu.

- utrwalenie pracy ze strukturami danych w C++;
- zastosowanie kontenera vector do przechowywania wielu rekordów;
- obsługa wejścia i wyjścia w aplikacji konsolowej;
- zapis i odczyt danych z pliku tekstowego;
- opracowanie podstawowej dokumentacji projektowej zgodnej z podejściem PBL.

3. Zakres systemu

Zakres systemu obejmuje funkcje dostępne z poziomu menu głównego programu:

- Dodanie nowego zgłoszenia usterki.
- Wyświetlenie skróconej listy wszystkich zgłoszeń.
- Wyświetlenie szczegółów zgłoszenia po podaniu ID.
- Zmiana statusu istniejącego zgłoszenia.
- Usunięcie zgłoszenia z listy.
- Wyświetlenie statystyk według statusów.
- Automatyczny zapis i odczyt danych z pliku tekstowego.

Poza zakresem obecnej wersji znajdują się: logowanie użytkowników, baza danych, panel administratora w przeglądarce, obsługa załączników oraz komunikacja sieciowa. Elementy te mogą zostać potraktowane jako kierunki dalszego rozwoju aplikacji.

4. Wymagania funkcjonalne i нефункционалне

4.1. Wymagania funkcjonalne

ID	Wymaganie	Opis
F1	Dodawanie zgłoszenia	Program pozwala wprowadzić tytuł,

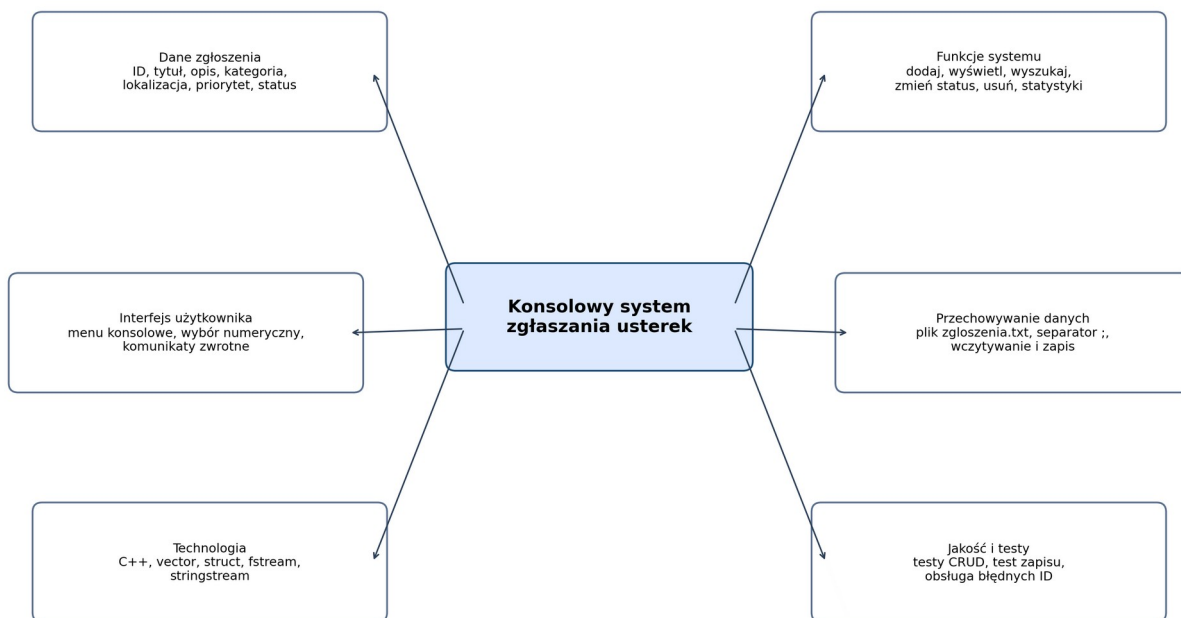
		opis, lokalizację, kategorię i priorytet usterki.
F2	Automatyczne ID	Każde zgłoszenie otrzymuje unikalny numer ID nadawany przez program.
F3	Lista zgłoszeń	Program wyświetla skróconą listę zgłoszeń wraz z najważniejszymi informacjami.
F4	Szczegóły zgłoszenia	Po podaniu ID użytkownik może zobaczyć pełne dane wskazanego zgłoszenia.
F5	Zmiana statusu	Program umożliwia zmianę statusu na: Nowe, Przyjęte, W trakcie naprawy, Naprawione lub Odrzucone.
F6	Usuwanie zgłoszenia	Program pozwala usunąć zgłoszenie o wskazanym ID.
F7	Statystyki	Program liczy zgłoszenia według statusów i pokazuje liczbę wszystkich pozycji.
F8	Trwałość danych	Program zapisuje dane do pliku i wczytuje je po ponownym uruchomieniu.

4.2. Wymagania niefunkcjonalne

ID	Wymaganie	Opis
NF1	Prostota obsługi	Menu powinno być czytelne i dostępne z poziomu konsoli.
NF2	Czytelność kodu	Kod powinien być podzielony na funkcje o jednoznacznych nazwach.
NF3	Niezawodność podstawowa	Program powinien informować o braku zgłoszeń lub nieistniejącym ID.
NF4	Przenośność	Kod powinien kompilować się w standardowym środowisku C++.
NF5	Możliwość rozbudowy	Struktura Zgłoszenie i funkcje programu powinny umożliwiać dodanie kolejnych pól lub filtrów.

5. Mapa myśli projektu

Mapa myśli przedstawia główne obszary aplikacji: dane zgłoszenia, funkcje użytkowe, sposób przechowywania danych, interfejs konsolowy, zastosowaną technologię oraz obszar testów.



Rys. 1. Mapa myśli przedstawiająca główne elementy projektu

Rys. 1. Mapa myśli projektu

6. Przypadki użycia

W obecnej wersji aplikacji występuje jeden aktor: użytkownik programu. Użytkownik wykonuje operacje dostępne z menu głównego.

ID	Nazwa	Scenariusz	Rezultat
PU1	Dodaj zgłoszenie	Użytkownik wybiera opcję 1, uzupełnia dane i zapisuje nowe zgłoszenie.	Nowy rekord zostaje dopisany do listy i pliku.
PU2	Wyświetl zgłoszenia	Użytkownik wybiera opcję 2.	Program pokazuje listę zgłoszeń albo komunikat o braku danych.
PU3	Wyszukaj po ID	Użytkownik wybiera opcję 3 i podaje numer ID.	Program wyświetla szczegóły albo komunikat o braku zgłoszenia.
PU4	Zmień status	Użytkownik wybiera opcję 4, podaje ID i wybiera nowy status.	Status zostaje zaktualizowany i zapisany w pliku.
PU5	Usuń zgłoszenie	Użytkownik wybiera opcję 5 i podaje ID.	Zgłoszenie zostaje usunięte z listy oraz z pliku.
PU6	Pokaż statystyki	Użytkownik wybiera opcję 6.	Program prezentuje liczbę zgłoszeń według statusów.

7. Model danych

Podstawową jednostką danych jest struktura Zgłoszenie. Reprezentuje ona pojedynczą usterkę w systemie.

```
struct Zgłoszenie {  
    int id;  
    string tytuł;  
    string opis;  
    string kategoria;  
    string lokalizacja;  
    string priorytet;  
    string status;  
};
```

Pole	Typ	Znaczenie
id	int	Unikalny identyfikator zgłoszenia nadawany automatycznie.
tytuł	string	Krótką nazwa problemu, np. Zepsute światło.
opis	string	Dokładniejszy opis usterki podany przez użytkownika.
kategoria	string	Typ usterki, np. Elektryka, Hydraulika, Meble.
lokalizacja	string	Miejsce występowania problemu, np. Sala 204.
priorytet	string	Ważność zgłoszenia: Niski, Średni, Wysoki, Pilny.
status	string	Etap obsługi zgłoszenia.

7.1. Format pliku danych

Dane są zapisywane w pliku tekstowym zgloszenia.txt. Każda linia odpowiada jednemu zgłoszeniu, a pola są rozdzielone średnikiem.

```
id;tytuł;opis;kategoria;lokalizacja;priorytet;status  
1;Zepsute światło;Nie działa światło;Elektryka;Sala 204;Sredni;Nowe
```

Zaletą takiego rozwiązania jest prostota implementacji. Ograniczeniem jest to, że wpisanie średnika w tytule, opisie lub lokalizacji może zaburzyć strukturę pliku. W wersji rozwojowej można zastosować format CSV z obsługą cudzysłówów albo plik JSON.

8. Architektura i opis kodu źródłowego

Program ma architekturę proceduralną. Dane są przechowywane globalnie w wektorze zgloszenia, natomiast kolejne operacje zostały wydzielone do funkcji. Dzięki temu kod jest czytelny i łatwy do zaprezentowania podczas obrony projektu.

```
vector<Zgłoszenie> zgloszenia;  
int nastepneId = 1;
```

Funkcja	Odpowiedzialność
wczytajBufor()	Czyści bufor wejścia po wczytaniu danych liczbowych.
wybierzKategorie()	Wyświetla listę kategorii i zwraca nazwę wybranej kategorii.

wybierzPriorytet()	Wyświetla listę priorytetów i zwraca wybrany priorytet.
wybierzStatus()	Wyświetla listę statusów i zwraca nowy status zgłoszenia.
dodajZgloszenie()	Tworzy nowe zgłoszenie, nadaje ID, pobiera dane od użytkownika i zapisuje rekord.
wyswietlZgloszenie()	Wyświetla szczegółowe dane jednego zgłoszenia.
wyswietlWszystkieZgloszenia()	Wyświetla skróconą listę wszystkich zgłoszeń.
znajdzIndeksPoid()	Przeszukuje wektor i zwraca indeks zgłoszenia o podanym ID albo -1.
wyszukajZgloszeniePoid()	Pobiera ID od użytkownika i wyświetla znalezione zgłoszenie.
zmienStatusZgloszenia()	Pozwala zmienić status wybranego zgłoszenia i zapisuje zmianę do pliku.
usunZgloszenie()	Usuwa zgłoszenie z wektora na podstawie ID.
pokazStatystyki()	Zlicza zgłoszenia według statusów.
zapiszDoPliku()	Zapisuje aktualny stan wektora do pliku tekstowego.
wczytajZPliku()	Wczytuje dane z pliku oraz wyznacza kolejne dostępne ID.
pokazMenu()	Wyświetla menu główne programu.
main()	Uruchamia program, wczytuje dane, obsługuje pętlę menu i kończy działanie.

9. Przebieg działania programu

Główna logika programu opiera się na pętli do-while, która działa do momentu wybrania opcji 0. Przed rozpoczęciem pętli program wczytuje dane z pliku. Po operacjach zmieniających dane następuje zapis do pliku.

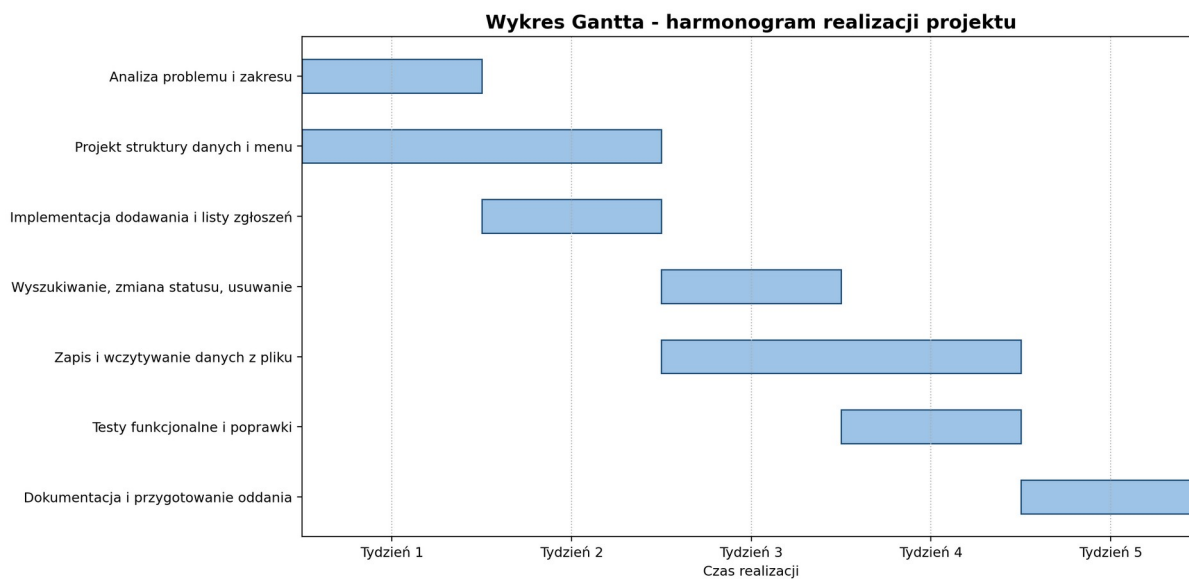
1. Start programu.
2. Wywołanie funkcji wczytajZPliku().
3. Wyświetlenie menu głównego.
4. Pobranie wyboru użytkownika.
5. Wykonanie odpowiedniej funkcji na podstawie instrukcji switch.
6. Powrót do menu albo zakończenie programu po wyborze 0.

9.1. Przykładowy scenariusz użycia

Krok	Akcja	Efekt
1	Użytkownik wybiera opcję 1.	Program przechodzi do dodawania zgłoszenia.
2	Użytkownik wpisuje tytuł, opis i lokalizację.	Dane trafiają do nowej struktury Zgloszenie.
3	Użytkownik wybiera kategorię oraz priorytet.	Program przypisuje wartości tekstowe na podstawie wyboru numerycznego.
4	Program nadaje status Nowe.	Zgłoszenie ma stan początkowy.
5	Program dodaje rekord do wektora i zapisuje plik.	Dane są zachowane po zamknięciu programu.

10. Wykres Gantta i harmonogram realizacji

Harmonogram przedstawia przykładowy plan realizacji projektu w modelu pięcioletniowym. Został dopasowany do zakresu aplikacji konsolowej i dokumentacji PBL.



Rys. 2. Przykładowy harmonogram projektu PBL w ujęciu pięcioletniowym

Rys. 2. Wykres Gantta projektu

Lp.	Etap	Opis	Termin
1	Analiza problemu i zakresu	Określenie funkcji systemu, zakresu i danych zgłoszenia.	Tydzień 1
2	Projekt struktury danych i menu	Zaprojektowanie struktury Zgłoszenie, list wyboru oraz menu.	Tydzień 1-2
3	Implementacja podstawowych operacji	Dodawanie oraz wyświetlanie zgłoszeń.	Tydzień 2
4	Implementacja operacji zarządzania	Wyszukiwanie po ID, zmiana statusu, usuwanie.	Tydzień 3
5	Obsługa pliku	Zapis i odczyt danych z pliku zgłoszenia.txt.	Tydzień 3-4
6	Testy i poprawki	Sprawdzenie działania funkcji i obsługi sytuacji brzegowych.	Tydzień 4
7	Dokumentacja	Opis projektu, mapa myśli, Gantt, testy i podsumowanie.	Tydzień 5

11. Plan testów

Testy mają potwierdzić poprawność najważniejszych funkcji systemu. Ze względu na konsolowy charakter aplikacji testy mogą zostać wykonane ręcznie na podstawie przygotowanych przypadków.

ID	Obszar testu	Kroki	Oczekiwany rezultat
T1	Dodanie zgłoszenia	Wprowadzić poprawny tytuł, opis, lokalizację, kategorię i priorytet.	Na liście pojawia się nowe zgłoszenie ze statusem Nowe.
T2	Wyświetlenie pustej listy	Uruchomić program bez pliku lub z pustym plikiem.	Program pokazuje komunikat Brak zgłoszeń.
T3	Wyszukanie istniejącego ID	Dodać zgłoszenie i wyszukać jego ID.	Program pokazuje szczegółowe dane zgłoszenia.
T4	Wyszukanie nieistniejącego ID	Podać ID, którego nie ma na liście.	Program pokazuje komunikat o braku zgłoszenia.
T5	Zmiana statusu	Zmienić status istniejącego zgłoszenia na Naprawione.	Status zgłoszenia zostaje zmieniony i zapisany.
T6	Usunięcie zgłoszenia	Usunąć zgłoszenie o istniejącym ID.	Zgłoszenie znika z listy.
T7	Statystyki	Dodać kilka zgłoszeń o różnych statusach.	Program poprawnie zlicza statusy.
T8	Trwałość danych	Dodać zgłoszenie, zamknąć program i uruchomić ponownie.	Zgłoszenie zostaje wczytane z pliku.

12. Analiza ryzyka

ID	Ryzyko	Opis	Poziom	Działanie zapobiegawcze
R1	Błędne dane wejściowe	Użytkownik wpisze tekst tam, gdzie oczekiwana jest liczba.	Średnie	Czyścić bufor i dodać dodatkową walidację wejścia.
R2	Uszkodzony format pliku	Plik zgłoszenia.txt zostanie ręcznie zmieniony lub będzie miał niepoprawną liczbę pól.	Średnie	Dodać kontrolę liczby pól i obsługę wyjątków stoi.
R3	Średnik w treści zgłoszenia	Znak ; w opisie może zaburzyć odczyt pól.	Niskie	Zastosować escapowanie znaków lub format JSON.
R4	Utrata danych przy zapisie	Błąd dostępu do pliku uniemożliwi zapis.	Niskie	Sprawdzać dostępność pliku i informować użytkownika.
R5	Ograniczona skalowalność	Przy dużej liczbie zgłoszeń liniowe wyszukiwanie może być wolniejsze.	Niskie	W przyszłości zastosować mapę lub bazę danych.

13. Instrukcja uruchomienia

Do uruchomienia programu potrzebny jest kompilator C++ obsługujący standard C++11 lub nowszy.

Przykładowa kompilacja przy użyciu g++:

```
g++ -std=c++17 main.cpp -o system_usterek  
./system_usterek
```

W systemie Windows przykładowe polecenia mogą wyglądać następująco:

```
g++ -std=c++17 main.cpp -o system_usterek.exe  
system_usterek.exe
```

Program tworzy i aktualizuje plik zgloszenia.txt w katalogu, z którego został uruchomiony. Jeżeli plik nie istnieje, aplikacja rozpoczyna pracę z pustą listą zgłoszeń.

14. Możliwości rozwoju

Obecny program spełnia założenia prostej aplikacji konsolowej. W przyszłości można go rozbudować o następujące funkcje:

- walidacja wejścia tak, aby program nie reagował błędnie na tekst wpisany zamiast liczby;
- filtrowanie zgłoszeń po kategorii, priorytecie lub statusie;
- sortowanie zgłoszeń według priorytetu albo numeru ID;
- zapis danych w formacie JSON lub CSV z obsługą znaków specjalnych;
- osobne role użytkownika, administratora i technika;
- historia zmian statusu zgłoszenia;
- wersja z bazą danych SQLite;
- interfejs graficzny albo aplikacja webowa.

15. Podsumowanie

Konsolowy system zgłaszania usterek jest kompletną, spójną aplikacją edukacyjną. Realizuje podstawowe operacje CRUD, pokazuje pracę z własną strukturą danych, wektorem, funkcjami oraz plikami tekstowymi. Projekt ma jasno określony problem, praktyczne zastosowanie i prostą architekturę, dzięki czemu dobrze pasuje do przedmiotu PBL.

Najważniejszą wartością projektu jest przedstawienie pełnego procesu obsługi zgłoszenia: dodanie problemu, zapis danych, późniejsze odnalezienie zgłoszenia, zmiana statusu oraz analiza liczby zgłoszeń. Pomimo prostoty aplikacja może stanowić bazę do dalszej rozbudowy, np. o bazę danych, role użytkowników lub interfejs graficzny.